

# Laboratório de Concorrência · Aula 10

Rômulo Amorim · RA 24000582-2 · Turma B

Gerado em 09/10/2026, 20:42:04 · tempo registrado no cronômetro: 67:37

## Preparação: Concorrência em cinco minutos

---

**0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.**

Cooperação: Basicamente uma tarefa depende da outra pra continuar, a 1 só vai avançar se a 2 tiver feito o que devia. Já a de competição é quando varias tarefas querem usar os mesmos recursos, pra evitar problema e conflito, precisam organizar esse uso.

Exemplos do dia a dia podem ser cooperação: Em um restaurante um chefe prepara a comida e o garçom leva até a mesa, a comida ja pronta, feita pelo chefe de cozinha.

De competição: Pode ser como em um banco, onde 2 pessoas querem e precisam usar um caixa eletrônico ao mesmo tempo, se elas não se organizarem e esperarem sua vez de usar, não vai dar certo.

## Prática 1: A condição de corrida

---

**Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?**

espero que 2.000.000

**1ª execução (n = 1.000.000)**

1508144

**2ª execução (n = 1.000.000)**

1352821

**3ª execução (n = 1.000.000)**

1022980

**Saída do programa**

esperado: 2000000

obtido: 1508144

esperado: 2000000

obtido: 1352821

esperado: 2000000

obtido: 1022980

a print anexada é quando troquei 1\_000\_000 por 1000 e executei mais duas vezes.

```

1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0;           // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1000;                 // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start();         // as duas executam ao mesmo tempo
14        t1.join(); t2.join();          // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
  
```

Console Output:

```

esperado: 2000
obtido: 2000
  
```

### 1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não bateram, deu diferente nas 3 execuções. Isso acontece porque as 2 threads alteram o mesmo contador ao mesmo tempo sem sincronizar. Algumas somas são perdidas e por isso os resultados mudam.

### 1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Primeiro, a Thread A lê o valor 10 do contador. Antes de gravar o resultado, a Thread B também lê o valor 10. Depois, as duas threads somam 1 ao valor que leram, chegando a 11. A Thread A grava 11 no contador e, em seguida, a Thread B também grava 11.

Assim, o contador fica com 11 em vez de 12, pois uma das atualizações foi perdida.

### 1.3 Com $n = 1000$ , o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com  $n = 1000$ , o resultado foi 2000 nas duas execuções, igual ao esperado. Isso aconteceu porque, com menos repetições, o erro pode não aparecer. Em sistemas reais, esse problema é perigoso porque pode passar despercebido durante os testes e acontecer em produção, causando resultados incorretos, como erros em saldos bancários ou perda de dados.

## Prática 2: Todas as respostas possíveis (Figura 13.1)

### Ordens testadas no simulador

AAABBB  $\rightarrow$  8

BBBAAA  $\rightarrow$  7

AABBAB  $\rightarrow$  6

BABBAA  $\rightarrow$  4

### Saída do programa

*não respondido*



## 2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São possíveis 20 ordens de execução e aparecem 4 resultados diferentes: 4, 6, 7 e 8. Das 20 ordens, 1 dá o resultado 7? Não, 9 ordens resultam em 8, que é o valor certo.

## 2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Uma ordem que resulta em 4 é BABBAA. Primeiro, a tarefa B lê o valor inicial 2 e calcula 4, mas ainda não grava. Depois, a tarefa A lê o valor 2, soma 1 e grava 3. A tarefa B grava o valor 4. Em seguida, a tarefa B lê 4 e calcula 8, mas antes de gravar, a tarefa A lê 4, soma 1 e grava 5. Por fim, a tarefa A termina, mas a tarefa B grava o valor 8.

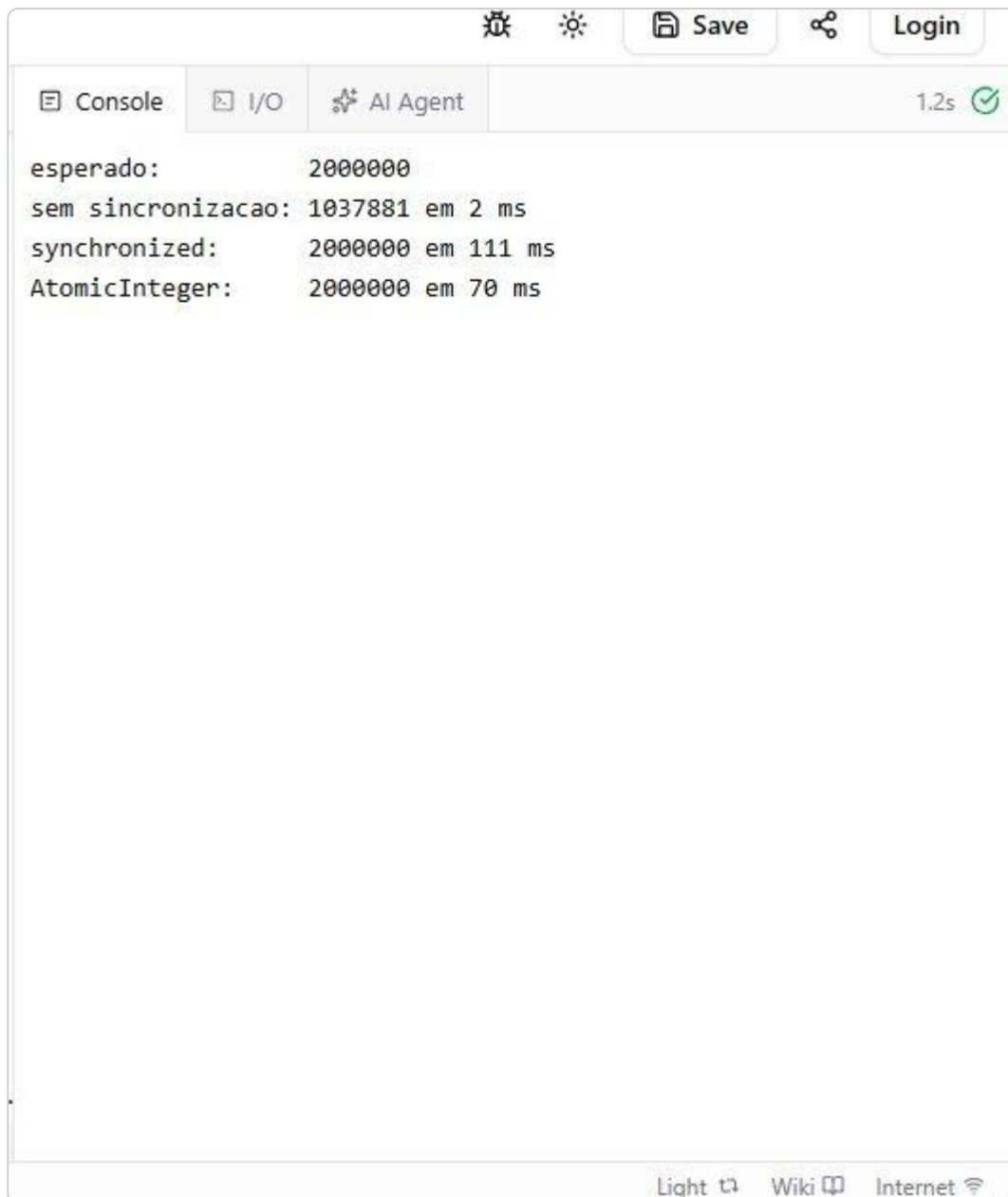
## Prática 3: Corrigindo a corrida

**Previsão:** As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acredito que a versão com AtomicInteger será a mais rápida, pois utiliza operações atômicas para atualizar o contador. Acho que nenhuma versão apresentará resultados incorretos, pois as três utilizam mecanismos de sincronização para evitar condições de corrida.

### Saída do programa

*não respondido*



The screenshot shows a web-based IDE interface. At the top, there are icons for a bug, a sun, a save button labeled 'Save', a share icon, and a 'Login' button. Below this is a tab bar with 'Console', 'I/O', and 'AI Agent'. The 'Console' tab is active, showing the following output:

```
esperado:          2000000
sem sincronizacao: 1037881 em 2 ms
synchronized:      2000000 em 111 ms
AtomicInteger:      2000000 em 70 ms
```

At the bottom right of the console, there is a status bar with 'Light' mode, a 'Wiki' link, and an 'Internet' connection indicator. A green checkmark and '1.2s' are visible in the top right corner of the console area.

```
8 esperado: 2000000
sem sincronizacao: 1047187 em 3 ms
synchronized: 2000000 em 149 ms
AtomicInteger: 2000000 em 24 ms
```

### 3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões `synchronized` e `AtomicInteger` deram o valor correto de 2.000.000 nas duas execuções. A versão `synchronized` levou 111 ms e 149 ms, enquanto `AtomicInteger` levou 70 ms e 24 ms. Já a versão sem sincronização levou apenas 2 ms e 3 ms, mas apresentou resultados incorretos. As versões corretas costumam ser mais lentas porque precisam controlar o acesso ao contador compartilhado para evitar condições de corrida e garantir que nenhuma soma seja perdida.

### 3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um `AtomicInteger` em cada saldo resolve? Por quê? O que seria preciso?

Não. Um `AtomicInteger` em cada saldo não garante que a retirada de uma conta e o depósito na outra aconteçam juntos. Seria necessário sincronizar a transferência inteira, garantindo que as duas operações sejam concluídas corretamente, sem deixar o dinheiro em um estado inconsistente.

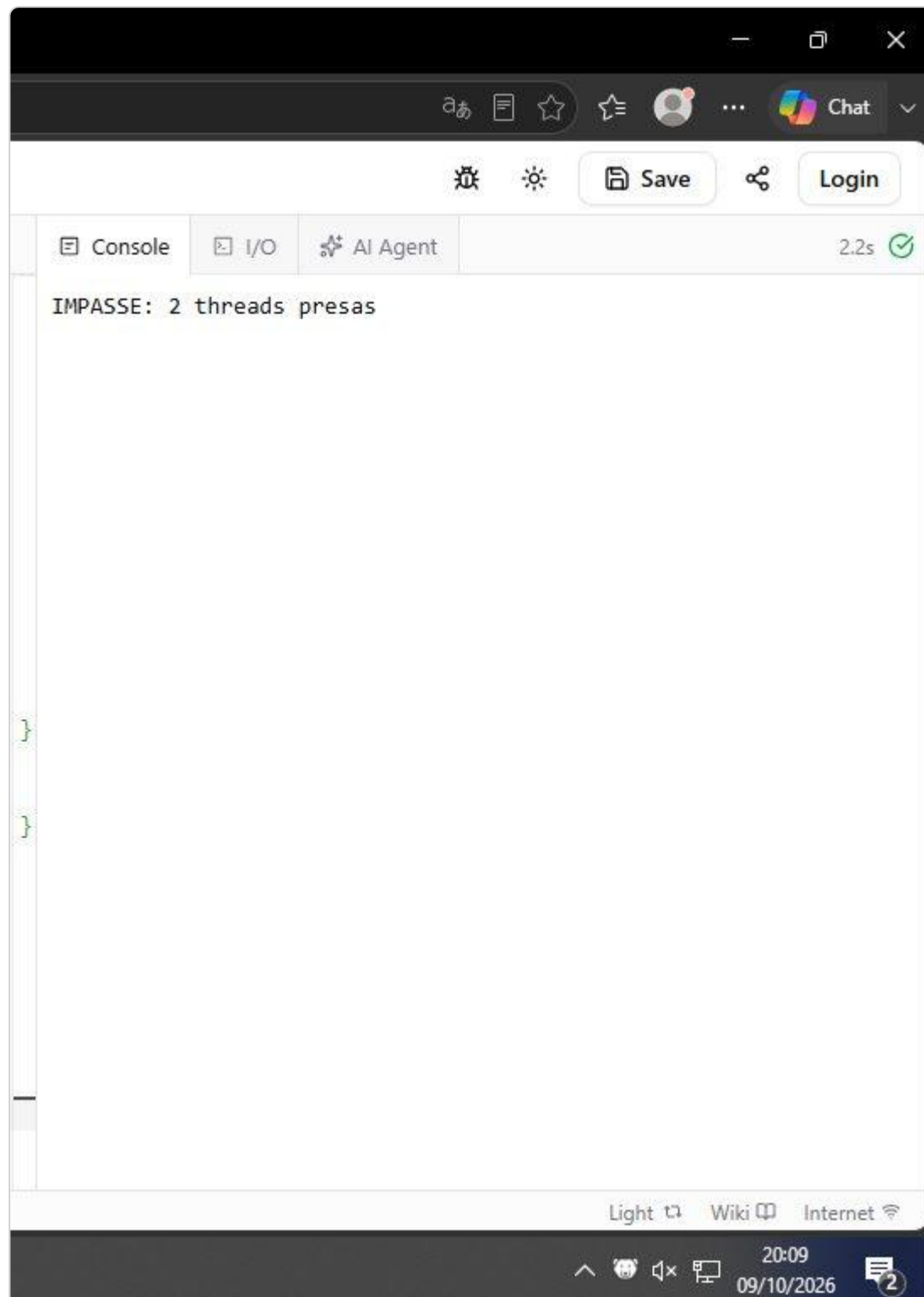
## Prática 4: Impasse (deadlock)

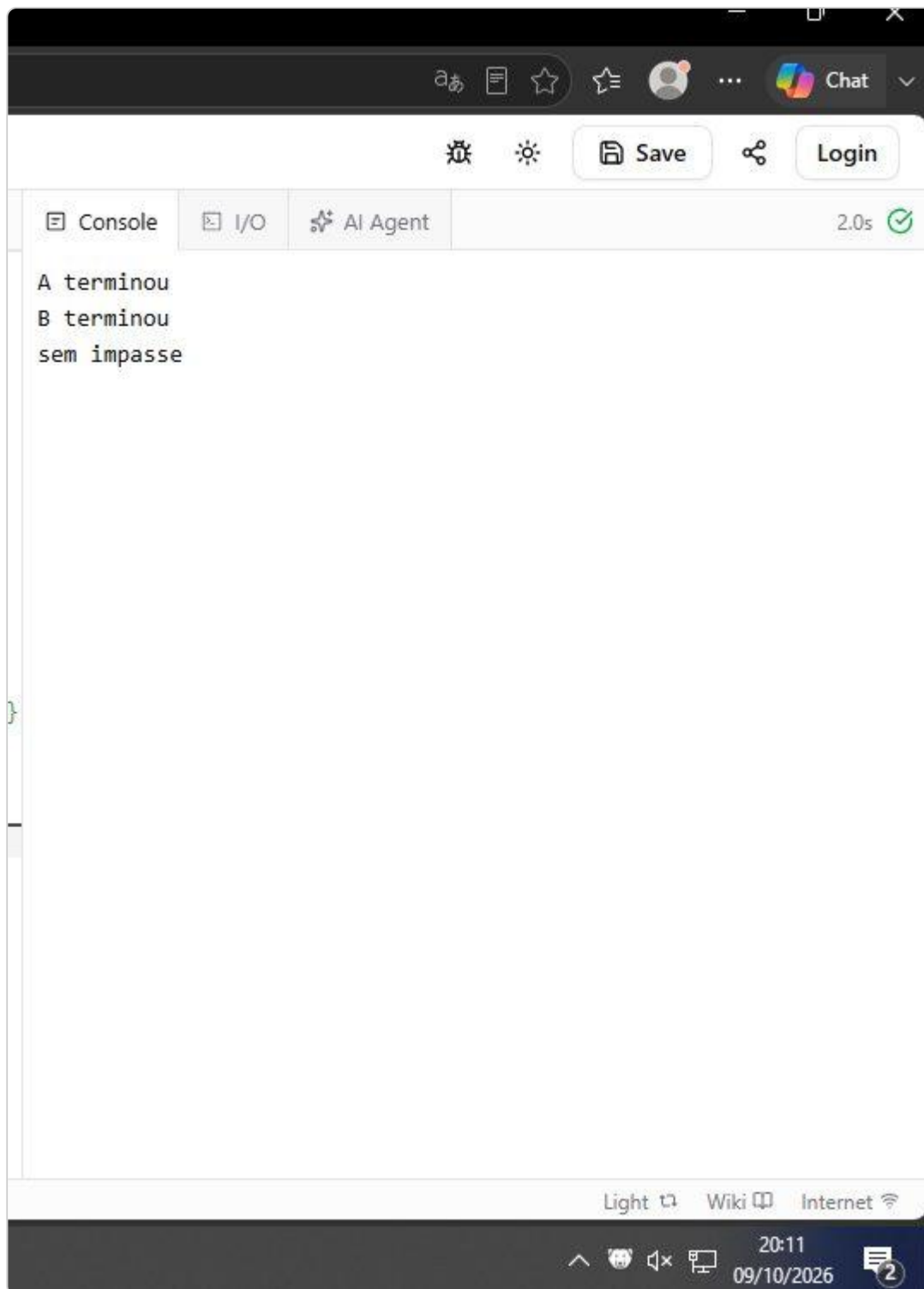
**Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?**

Acredito que as mensagens "A terminou" e "B terminou" não vão aparecer, porque a thread A segura o recurso X e espera pelo Y, enquanto a thread B segura o recurso Y e espera pelo X. Assim, uma fica esperando a outra liberar o recurso, causando um impasse deadlock.

**Saídas (antes e depois da correção)**

*não respondido*





#### 4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As duas threads ficaram presas porque a thread A tinha o recurso X e esperava pelo recurso Y, enquanto a thread B tinha o recurso Y e esperava pelo recurso X. Como cada uma estava segurando um recurso que a outra precisava, nenhuma conseguia continuar, causando um impasse (deadlock).

#### 4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pegar os recursos sempre na mesma ordem evita o impasse porque impede que uma thread fique esperando um recurso que está preso em outra, enquanto a outra espera pelo primeiro. Um exemplo real seria um sistema bancário em que duas transferências precisam acessar duas contas ao mesmo tempo. Se todas acessarem as contas na mesma ordem, o risco de deadlock é reduzido.

## Prática 5: Mensagens em vez de memória compartilhada

---

**Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?**

Com 3 trabalhadores, espero que leve aproximadamente 200 ms, pois cada trabalhador executará 2 tarefas de 100 ms. Com 1 trabalhador, espero cerca de 600 ms, pois ele fará todas as tarefas em sequência. Com 6 trabalhadores, espero aproximadamente 100 ms, pois cada trabalhador poderá executar uma tarefa ao mesmo tempo.

### Tempo com 1 trabalhador

600ms

### Tempo com 3 trabalhadores

200ms

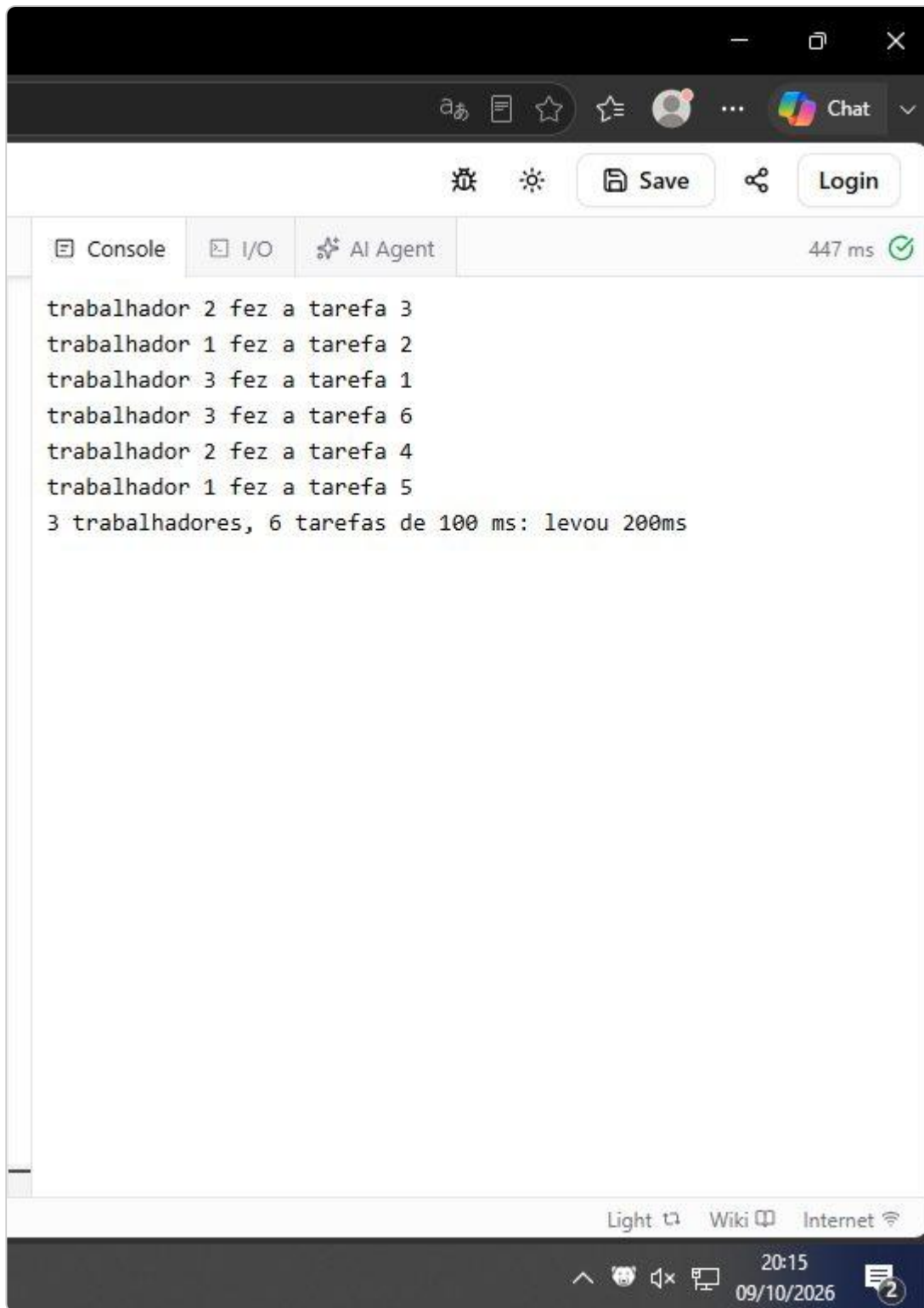
### Tempo com 6 trabalhadores

100ms

### Saída do programa

*não respondido*

```
trabalhador 1 fez a tarefa 1
trabalhador 5 fez a tarefa 5
trabalhador 6 fez a tarefa 6
trabalhador 3 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 4 fez a tarefa 4
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

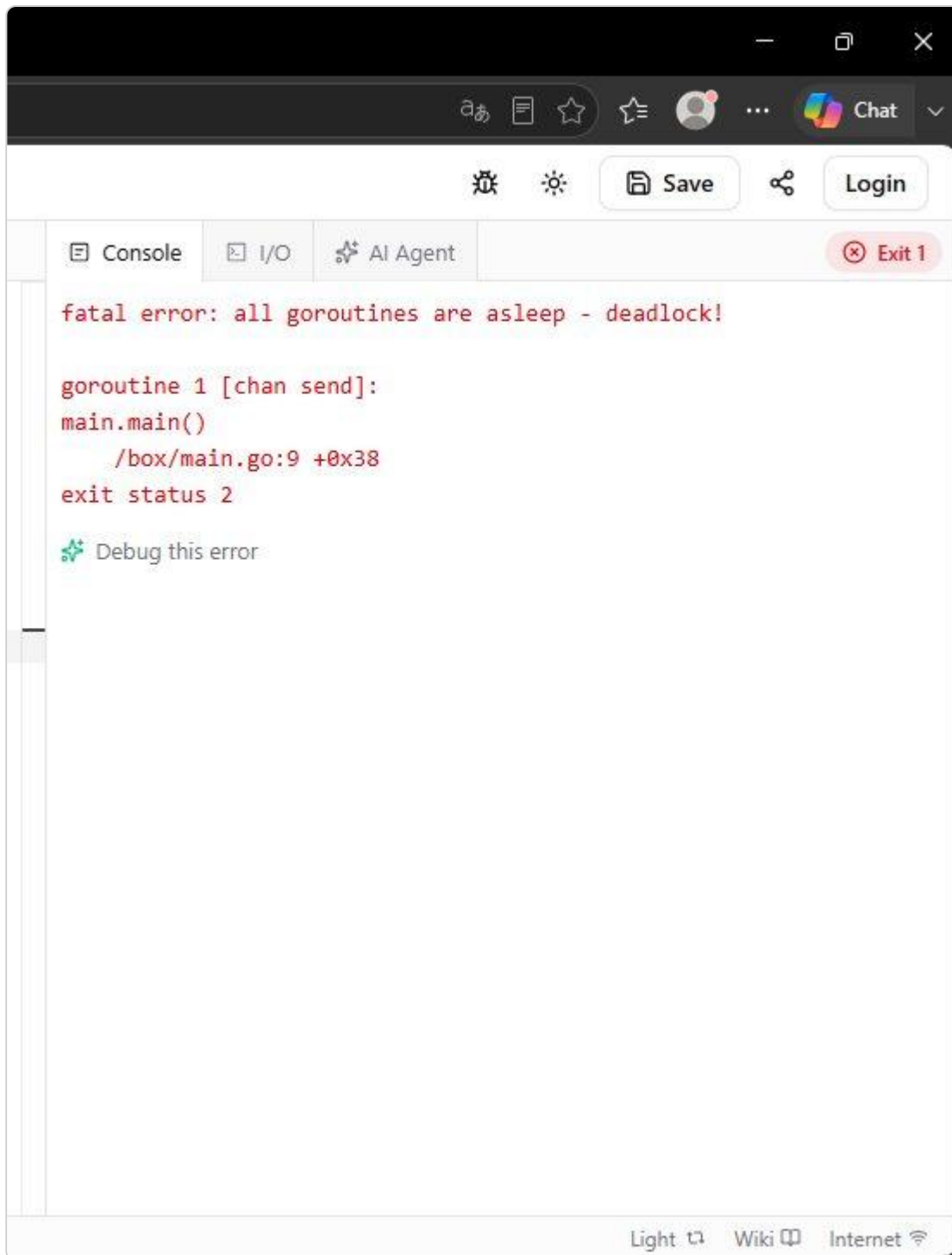


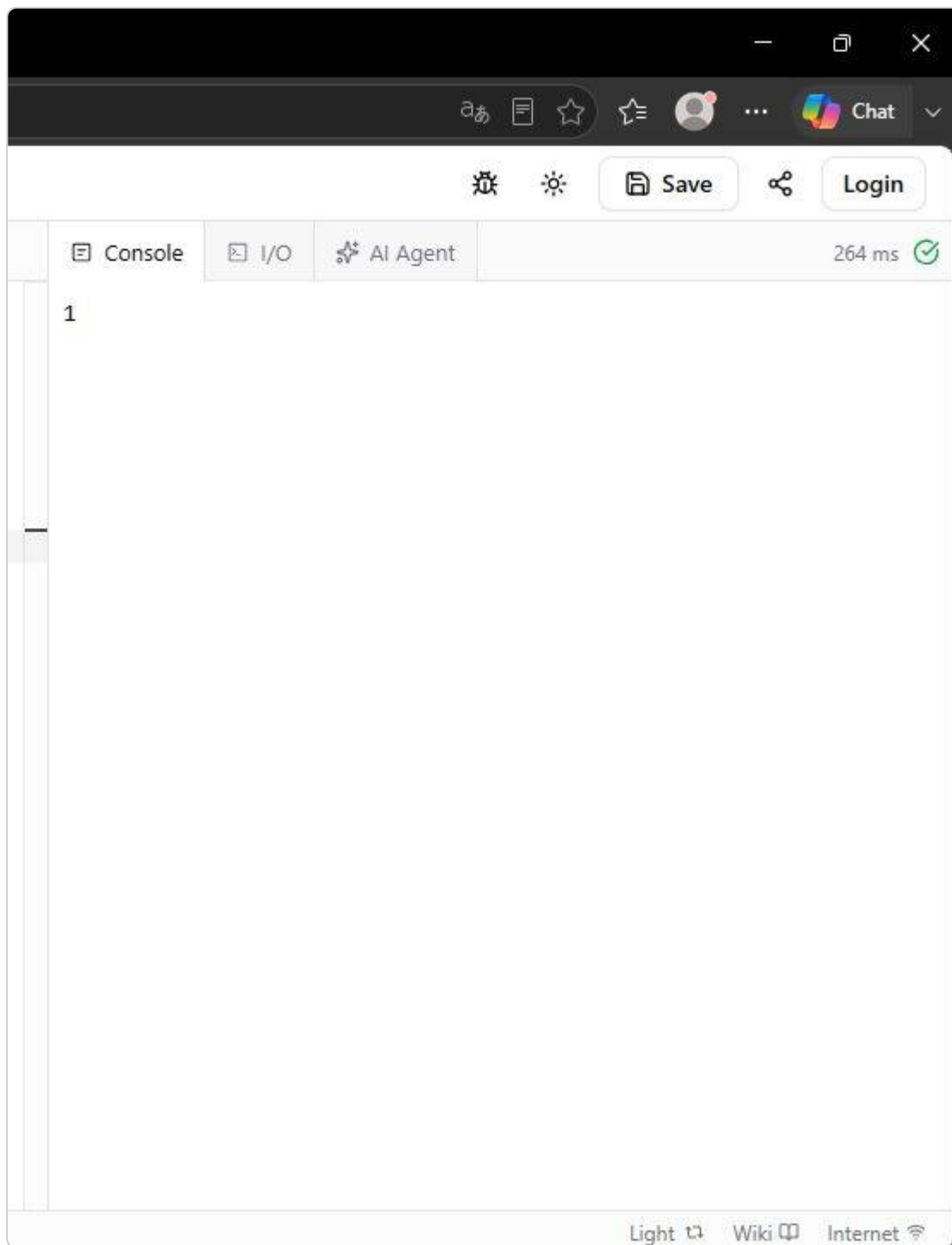
### 5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador, o tempo é maior porque as seis tarefas são executadas em sequência, levando cerca de 600 ms. Com 3 trabalhadores, o tempo cai para aproximadamente 200 ms, pois cada um realiza duas tarefas. Com 6 trabalhadores, o tempo esperado é próximo de 100 ms, pois cada trabalhador pode executar uma tarefa ao mesmo tempo. O programa não precisa de synchronized nem de travas porque as tarefas são enviadas por um canal, e cada trabalhador recebe uma tarefa para executar, sem precisar alterar um contador compartilhado. Isso reduz o risco de condições de corrida.

#### Saídas (antes e depois da troca)

*não respondido*





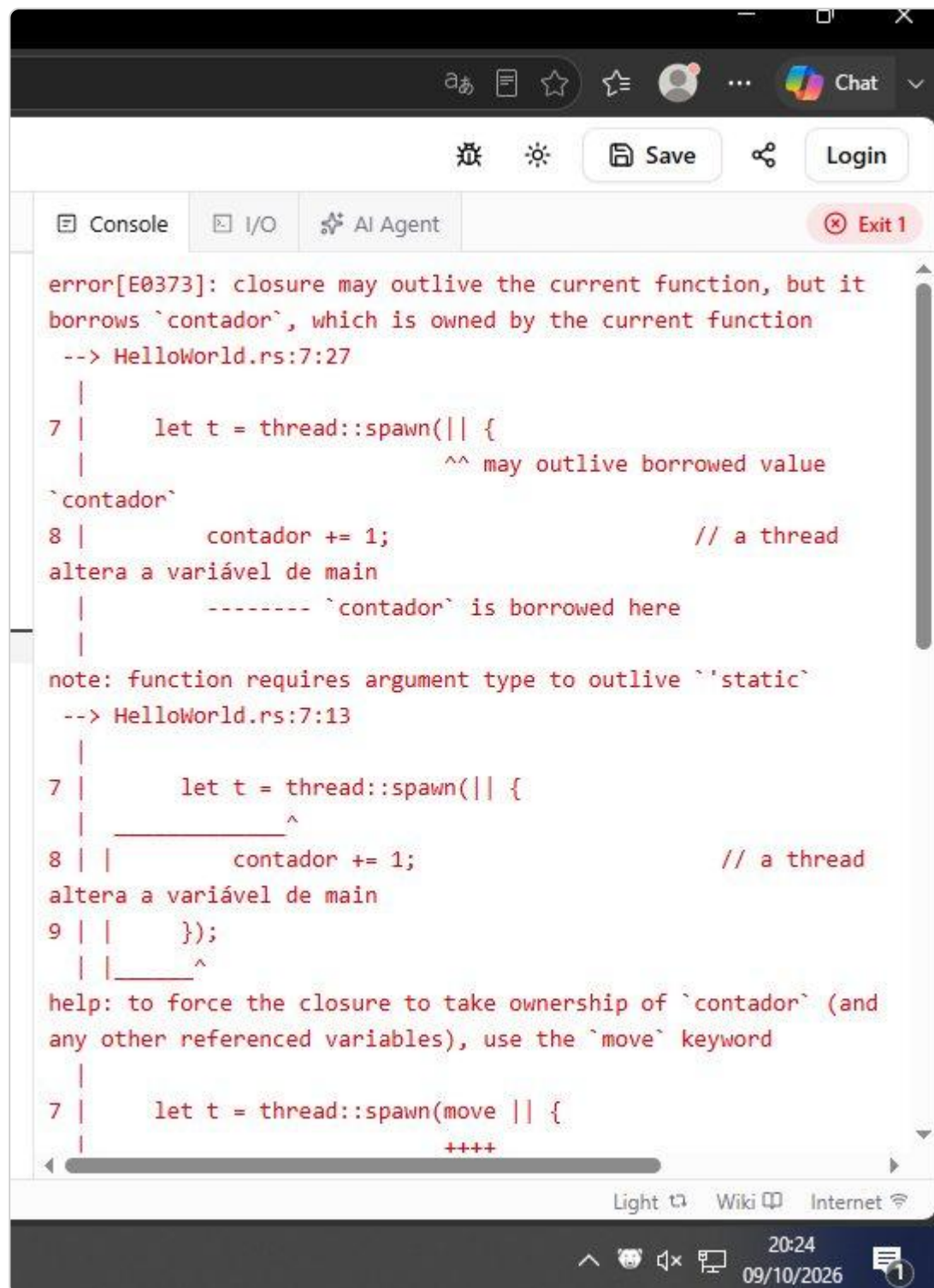
## 5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque o canal não tem espaço para armazenar mensagens, então o envio fica esperando alguém receber o valor. Como não existe outra goroutine para receber, ocorre um deadlock. Ao adicionar espaço para uma mensagem no canal, o valor pode ser armazenado sem precisar de um receptor imediato, permitindo que o programa continue e imprima o número 1.

## Prática 6: O compilador como guarda

**Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?**

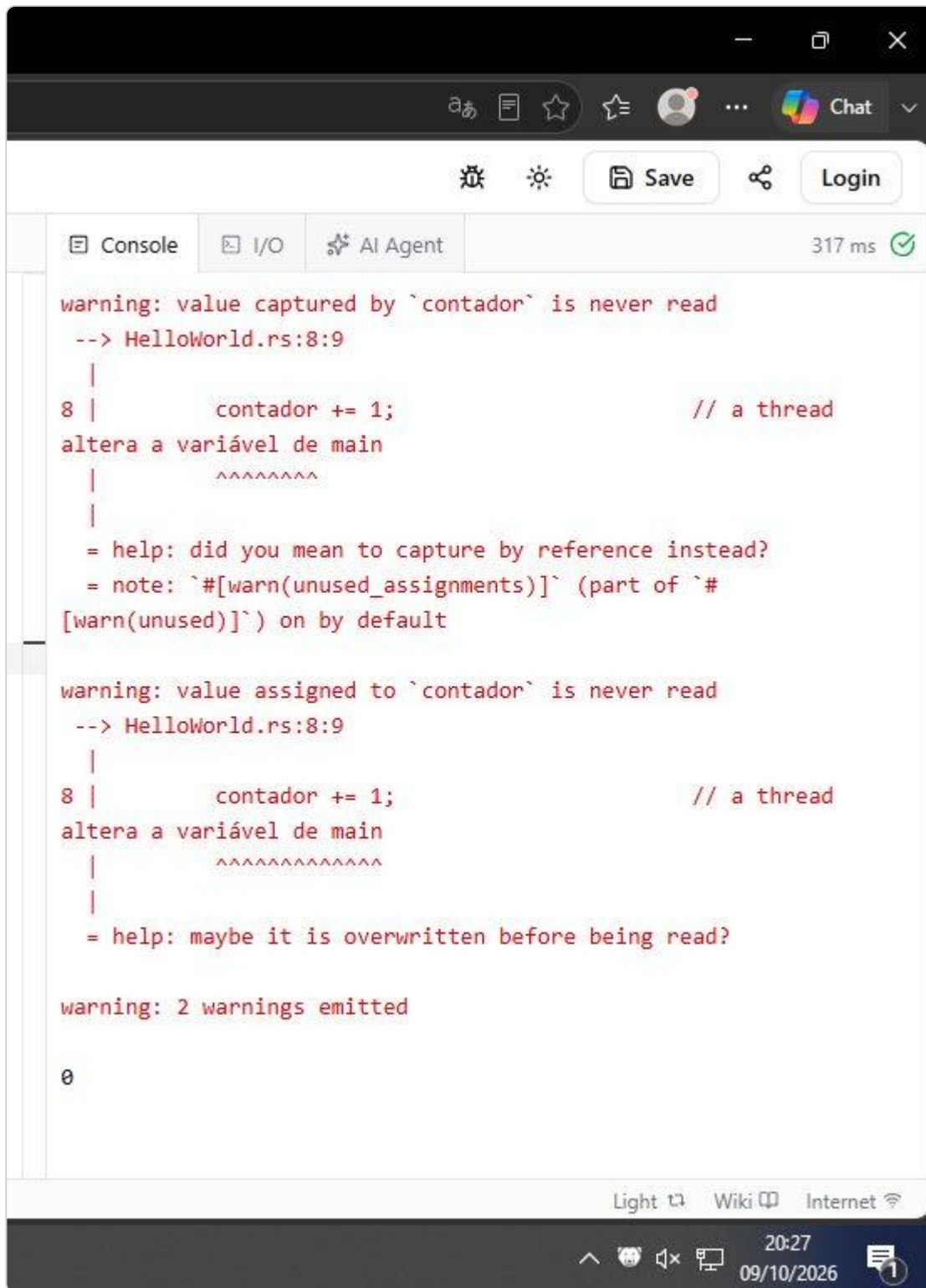
Acredito que o programa não vai compilar, porque o Rust não permite que uma thread altere diretamente uma variável que pertence à função main sem a proteção adequada. O compilador deve apresentar um erro, pois é necessário usar mecanismos seguros de compartilhamento de dados entre threads.

**Saídas (o erro e o resultado com move)***não respondido*

The screenshot shows a web-based IDE interface. At the top, there's a browser-like header with a search bar, a star icon, a user profile icon, and a 'Chat' button. Below this is a toolbar with icons for settings, a 'Save' button, a share icon, and a 'Login' button. The main area is divided into tabs: 'Console', 'I/O', and 'AI Agent'. The 'Console' tab is active, displaying the following text:

```
error[E0373]: closure may outlive the current function, but it
borrows `contador`, which is owned by the current function
--> HelloWorld.rs:7:27
|
7 |     let t = thread::spawn(|| {
|                               ^^ may outlive borrowed value
`contador`
8 |         contador += 1;           // a thread
altera a variável de main
|         ----- `contador` is borrowed here
|
note: function requires argument type to outlive `static`
--> HelloWorld.rs:7:13
|
7 |     let t = thread::spawn(|| {
|           ^
8 |         contador += 1;           // a thread
altera a variável de main
9 |     });
|     ^
help: to force the closure to take ownership of `contador` (and
any other referenced variables), use the `move` keyword
7 |     let t = thread::spawn(move || {
|                               +++++
```

At the bottom of the IDE, there's a status bar with 'Light' theme, 'Wiki' link, 'Internet' status, and a system tray showing the time '20:24', date '09/10/2026', and a notification icon with the number '1'.



The screenshot shows a web-based IDE interface. At the top, there's a browser-like header with a search bar, a star icon, a star icon, a user profile icon, a menu icon, and a 'Chat' button. Below this is a toolbar with a 'Save' button and a 'Login' button. The main area is a console window with tabs for 'Console', 'I/O', and 'AI Agent'. The 'Console' tab is active, showing two warning messages and some code. The first warning is 'warning: value captured by `contador` is never read' followed by '--> HelloWorld.rs:8:9'. Below this is a code snippet: '8 | contador += 1; // a thread altera a variável de main'. The second warning is 'warning: value assigned to `contador` is never read' followed by '--> HelloWorld.rs:8:9'. Below this is the same code snippet. The third warning is 'warning: 2 warnings emitted'. The console also shows a '0' at the bottom. The bottom of the IDE shows a status bar with 'Light', 'Wiki', and 'Internet' icons, and a system tray with a clock showing '20:27' and '09/10/2026'.

```
warning: value captured by `contador` is never read
--> HelloWorld.rs:8:9
|
8 |         contador += 1; // a thread
altera a variável de main
|         ^^^^^^^^^
|
= help: did you mean to capture by reference instead?
= note: `[warn(unused_assignments)]` (part of `[warn(unused)]`) on by default

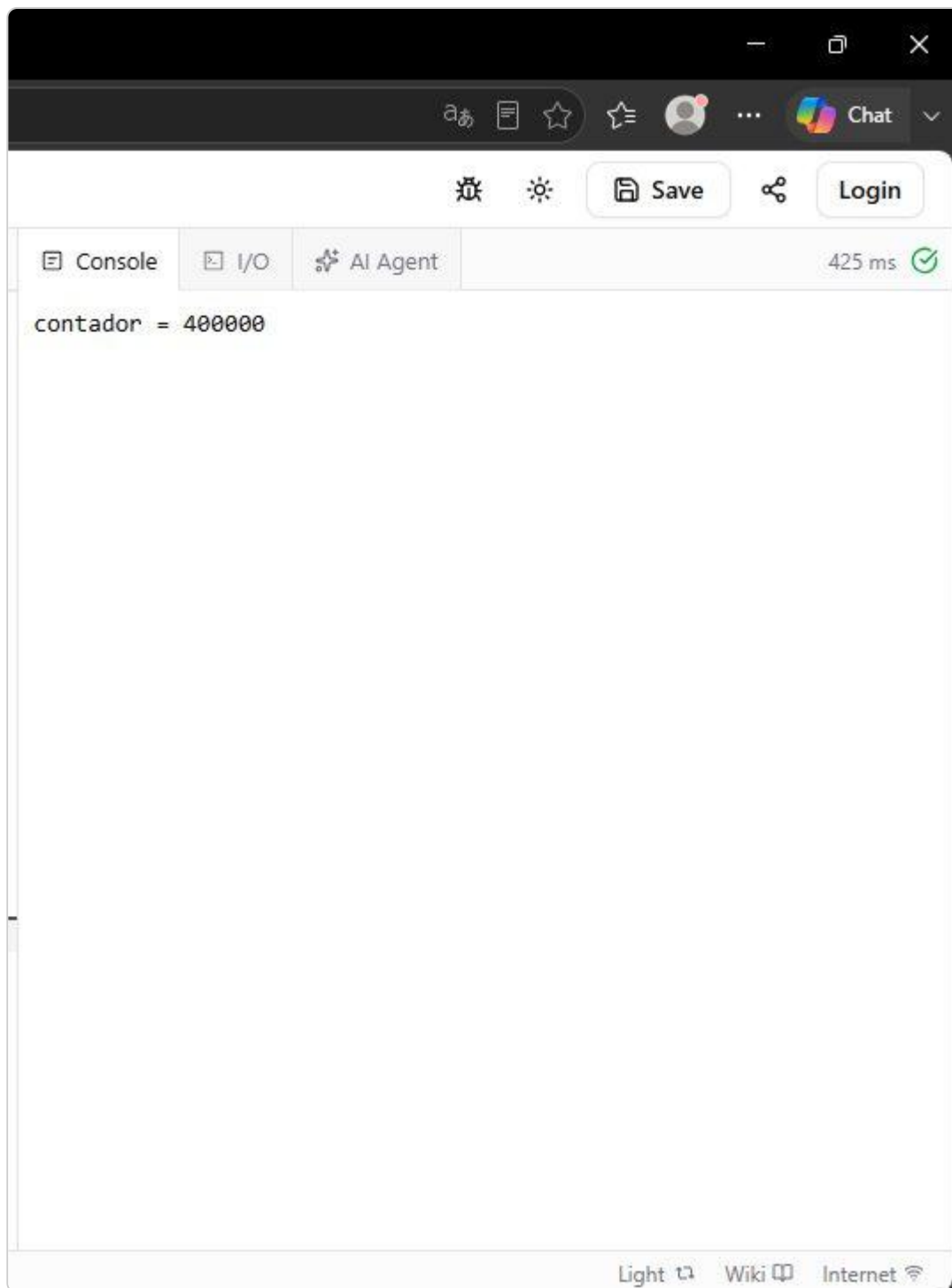
warning: value assigned to `contador` is never read
--> HelloWorld.rs:8:9
|
8 |         contador += 1; // a thread
altera a variável de main
|         ^^^^^^^^^^^^^^^
|
= help: maybe it is overwritten before being read?

warning: 2 warnings emitted

0
```

## Saída do programa

*não respondido*



### 6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com o move, a thread recebe a propriedade de uma cópia do valor da variável contador. Ela altera essa cópia, mas o valor original da função main continua sendo 0. Por isso, o programa imprime 0, mesmo que a thread tenha incrementado sua própria cópia.

### 6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o erro de concorrência foi percebido durante a execução do programa, quando o contador apresentou valores menores que o esperado. Já em Rust, o compilador identificou o problema antes da execução, ao tentar compartilhar e modificar a variável entre threads sem a proteção adequada.

A vantagem do Rust é que ele ajuda a evitar erros de concorrência antes mesmo de executar o programa,

aumentando a segurança. O custo é que o programador precisa aprender e utilizar regras e recursos específicos, como Arc e Mutex, o que pode tornar o código mais complexo.

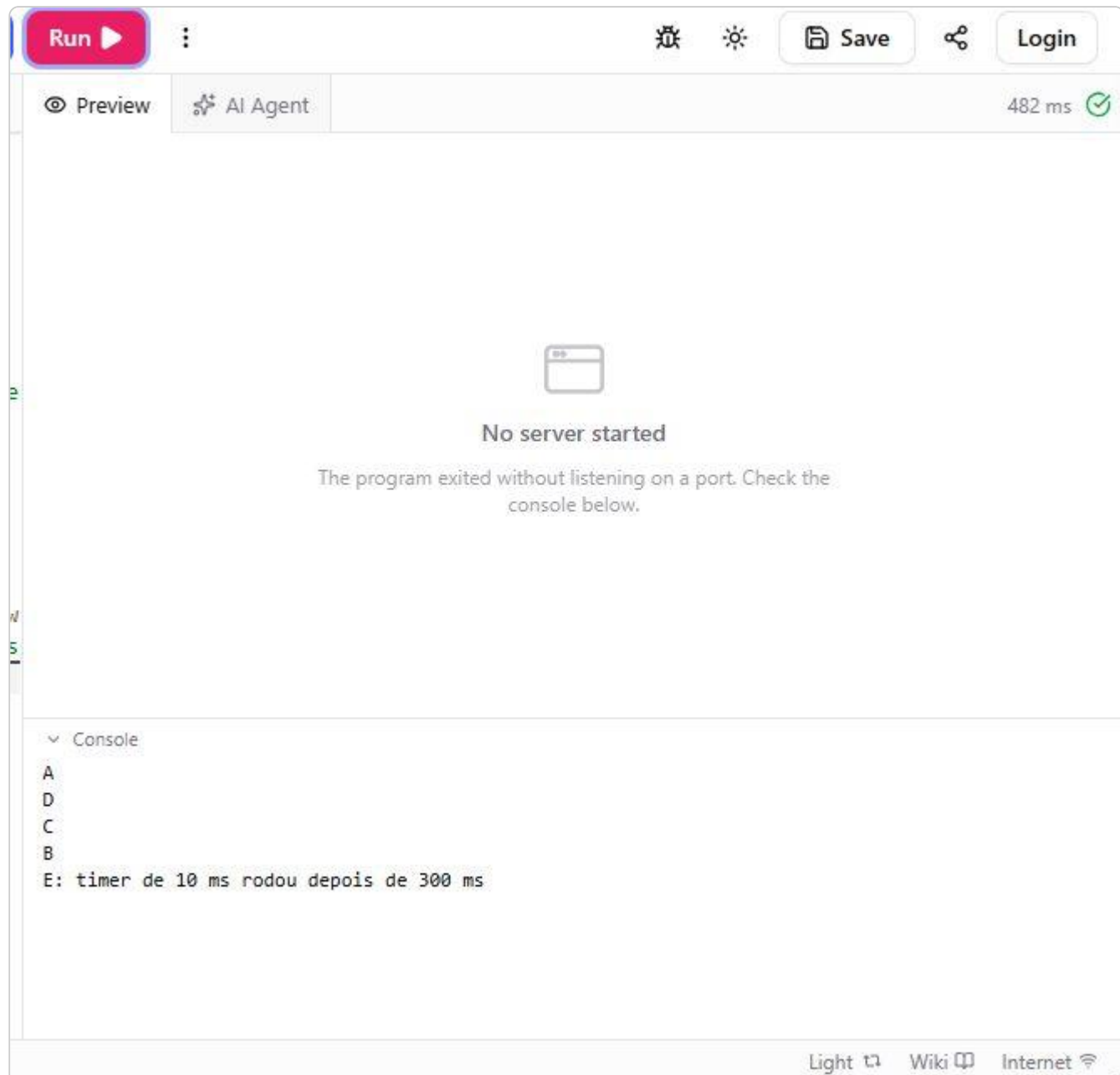
## Prática 7: Uma thread só: o laço de eventos

**Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?**

Acredito que as letras serão impressas na ordem A, D, C e B.

**Saída do programa**

*não respondido*



### 7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem de impressão foi A, D, C, B e E. A letra B, mesmo com tempo de 0 ms, saiu depois de D e C porque o `setTimeout` agenda a execução para depois que o código atual terminar. Já a promessa que imprime C é processada antes dos temporizadores. Portanto, o tempo zero não significa execução imediata. A letra E saiu por último porque o laço ocupado(300) bloqueou a única thread por aproximadamente 300 ms.

## **7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?**

O timer de 10 ms demorou aproximadamente 300 ms porque a função ocupado(300) bloqueou a única thread enquanto realizava um cálculo. Durante esse tempo, o Node.js não conseguiu executar outras tarefas do laço de eventos. Isso significa que, em um servidor que atende muitos usuários, um cálculo pesado pode atrasar as respostas de várias requisições, deixando o sistema lento. Por isso, é importante evitar bloquear a thread principal com operações demoradas.

## **Fechamento: Quem impede a corrida?**

---

### **Java sem sincronização**

Ninguém: há corrida

### **Java com synchronized**

O programador, com sincronização

### **Go com canais**

O projeto: não há memória compartilhada

### **Rust com Arc e Mutex**

O compilador

### **JavaScript**

Só existe uma thread

## **8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?**

Pode ocorrer uma condição de corrida: as duas pessoas podem verificar ao mesmo tempo que existe um ingresso disponível e ambas concluir a compra, vendendo mais ingressos do que há lugares. Para evitar isso, o sistema deve controlar o acesso ao estoque e realizar a verificação e a reserva do ingresso como uma única operação atômica, garantindo que apenas uma pessoa consiga comprar o último lugar.

## **8.2 Em uma frase: o que mais surpreendeu você nesta atividade?**

O que mais me surpreendeu foi perceber que programas podem apresentar resultados incorretos por causa da concorrência e que cada linguagem possui maneiras diferentes de evitar esses problemas.